Django Complete Notes — Beginner to Advanced

Python Django Backend Development Master Notes

Level: Beginner to Advanced

Language: English technical notes with easy explanations

Framework: Django + Django REST Framework

Purpose: Learning, practical projects, backend development, and interview preparation

---

TABLE OF CONTENTS

1. Django Introduction
2. Django Installation and Setup
3. Django Project Structure
4. URL Dispatcher
5. Function-Based Views (FBV)
6. Class-Based Views (CBV)
7. Django Templates
8. Template Inheritance
9. Static Files and Media Files
10. Models and Database
11. Django ORM
12. CRUD Operations
13. Forms and ModelForms
14. Django Admin
15. Authentication and Authorization
16. Sessions and Cookies
17. Middleware
18. Django REST Framework (DRF)
19. Serializers
20. APIView, Generic Views, and ViewSets
21. API Authentication
22. Permissions and Throttling
23. Filtering, Searching, and Pagination
24. Model Relationships

25. Query Optimization
26. Signals
27. Custom Managers and QuerySets
28. Testing
29. Security Best Practices
30. Caching
31. Background Tasks with Celery
32. Environment Variables and Settings
33. Deployment
34. Git and GitHub Workflow
35. Common Errors and Debugging
36. Interview Questions and Answers
37. Practice Projects
38. Learning Roadmap and Checklist

CHAPTER 1: DJANGO INTRODUCTION

1.1 What Is Django?

Django is a high-level Python web framework used to build secure, maintainable, and scalable web applications quickly.

Django provides many built-in features, so developers do not need to build everything from scratch.

Examples of applications you can build with Django:

- Blog websites
- E-commerce applications
- REST APIs
- Authentication systems
- Learning management systems
- Booking systems
- Social networking applications
- Admin dashboards
- SaaS applications

1.2 Why Use Django?

Advantages of Django:

1. **Fast Development:** Many common features are built in.
2. **Security:** Includes protection against several common web vulnerabilities.
3. **ORM:** Work with databases using Python code.
4. **Admin Panel:** Manage database records through a built-in interface.
5. **Authentication:** Provides user accounts, login, logout, groups, and permissions.
6. **Scalability:** Suitable for projects ranging from small applications to large services.
7. **Reusable Apps:** Organize functionality into separate applications.
8. **Migrations:** Track database schema changes.
9. **Testing Support:** Includes tools for automated testing.
10. **Large Ecosystem:** Integrates with tools such as Django REST Framework and Celery.

1.3 Django Architecture: MVT

Django follows the **MVT (Model–View–Template)** architectural pattern.

Model

The Model represents data and defines database operations.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()

    def __str__(self):
        return self.name
```

View

The View handles requests, performs application logic, and returns responses.

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django")
```

Template

The Template controls how data is presented to users.

```
<h1>Welcome to Django</h1>
<p>This is my first Django page.</p>
```

MVT Request Flow

1. The browser sends an HTTP request.
2. Django's URL dispatcher finds the matching URL pattern.
3. Django calls the associated View.
4. The View may retrieve data through the Model.
5. The View returns an HTTP response or renders a Template.
6. The browser displays the response.

Important: In Django terminology, the View handles request logic, while the Template handles presentation.

1.4 Django vs Flask vs FastAPI

Feature	Django	Flask	FastAPI
Main use	Full web applications	Lightweight web apps	APIs and async services
Built-in ORM	No single mandatory ORM, but Django ORM included	No	No
Built-in admin	Yes	No	No
Built-in authentication	Yes	Extensions commonly used	Extensions commonly used

API development	Django REST Framework	Extensions	Built-in API-focused features
Learning style	Many integrated components	Minimal core	Type hints and API-focused
Best fit	Full-featured applications	Flexible small applications	Modern APIs and async workloads

1.5 Important Django Terminology

- **Project:** The overall Django website and its configuration.
- **App:** A reusable component within a project.
- **Model:** Defines data structure and database behavior.
- **View:** Handles requests and responses.
- **Template:** Defines HTML presentation.
- **URLconf:** Maps URL paths to views.
- **ORM:** Object-Relational Mapper.
- **Migration:** A versioned database schema change.
- **Middleware:** Processes requests and responses globally.
- **QuerySet:** Represents a collection of database queries.
- **Admin:** Built-in interface for trusted staff to manage records.

Chapter 1 Interview Questions

Q1. What is Django?

Django is a high-level Python web framework used to develop secure and maintainable web applications.

Q2. What is MVT?

MVT stands for Model, View, and Template.

Q3. What is a Django app?

A Django app is a component that implements a specific part of a project.

Q4. Why is Django called a batteries-included framework?

Because it provides many built-in tools, including an ORM, authentication, admin, templates, forms, and security protections.

CHAPTER 2: DJANGO INSTALLATION AND SETUP

2.1 Prerequisites

Before learning Django, understand:

- Python fundamentals
- Functions and classes
- Lists, dictionaries, and loops
- Modules and packages
- Exception handling
- Basic HTML and CSS
- Basic SQL and HTTP concepts

2.2 Check Python Installation

```
python --version
python -m pip --version
```

Depending on your system, you may need `python3` instead of `python`.

2.3 Create a Project Folder

```
mkdir django_learning
cd django_learning
```

2.4 Create a Virtual Environment

A virtual environment isolates project dependencies.

```
python -m venv .venv
```

Windows activation:

```
.venv\Scripts\activate
```

Linux/macOS activation:

```
source .venv/bin/activate
```

2.5 Install Django

```
python -m pip install django
```

Check the installed version:

```
python -m django --version
```

2.6 Create a Django Project

```
django-admin startproject config .
```

The dot means the project files are created in the current directory.

Run the development server:

```
python manage.py runserver
```

Open the following URL in your browser:
`http://127.0.0.1:8000/`
Important: `runserver` is a development server. Do not use it as the production web server.

2.7 Create a Django App

```
python manage.py startapp core
```

Register the app in `config/settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "core",
]
```

2.8 Database Migrations

Run:

```
python manage.py makemigrations
python manage.py migrate
```

`makemigrations`: Creates migration files based on model changes.

`migrate`: Applies migrations to the database.

2.9 Create an Admin User

```
python manage.py createsuperuser
```

Follow the prompts to set up the account.

Then visit:
`http://127.0.0.1:8000/admin/`

2.10 Useful Django Commands

```
# Run development server
python manage.py runserver

# Create an app
python manage.py startapp core

# Create migration files
python manage.py makemigrations

# Apply migrations
python manage.py migrate

# Create admin user
python manage.py createsuperuser

# Open Django shell
python manage.py shell

# Run tests
python manage.py test

# Check project configuration
python manage.py check

# Production deployment checks
python manage.py check --deploy

# Collect static files
python manage.py collectstatic
```

Chapter 2 Interview Questions

Q1. Why use a virtual environment?

It isolates project dependencies and helps avoid conflicts between projects.

Q2. What is the difference between `makemigrations` and `migrate`?

`makemigrations` creates migration files; `migrate` applies them to the database.

Q3. What does `startproject` do?

It creates the initial Django project configuration.

Q4. What does `startapp` do?

It creates the initial structure for a Django application.

CHAPTER 3: DJANGO PROJECT STRUCTURE

3.1 Standard Project Structure

```
django_learning/
│
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── core/
│   ├── migrations/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
│
└── .venv/
```

3.2 `manage.py`

This file runs Django management commands.

```
python manage.py runserver
python manage.py migrate
```

```
python manage.py shell
```

3.3 `settings.py`

Contains project configuration, including:

- `INSTALLED_APPS`
- `MIDDLEWARE`
- `DATABASES`
- `TEMPLATES`
- `STATIC_URL`
- `ALLOWED_HOSTS`
- `SECRET_KEY`
- `DEBUG`

3.4 `urls.py`

Defines URL patterns and maps paths to views.

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]
```

3.5 `models.py`

Defines database models.

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

3.6 `views.py`

Defines request-handling logic.

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Home Page")
```

3.7 `admin.py`

Registers models with Django admin.

```
from django.contrib import admin
from .models import Category

admin.site.register(Category)
```

3.8 `apps.py`

Contains application configuration.

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "core"
```

3.9 `tests.py`

Contains automated tests.

```
from django.test import TestCase

class ExampleTest(TestCase):
    def test_basic_assertion(self):
        self.assertEqual(2 + 2, 4)
```

3.10 `wsgi.py` and `asgi.py`

- **WSGI:** Interface commonly used for synchronous Python web applications.
- **ASGI:** Interface supporting asynchronous applications and protocols.

Best Practice

As a project grows, divide functionality into meaningful apps such as:

```
accounts/
products/
orders/
payments/
notifications/
```

Keep each app focused on a clear responsibility.

CHAPTER 4: URL DISPATCHER

4.1 What Is URL Routing?

URL routing maps a requested URL path to a Django view.

For example:

`/about/` → `about` view

4.2 Project-Level URLs

`config/urls.py`

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("core.urls")),
]
```

4.3 App-Level URLs

Create `core/urls.py`:

```
from django.urls import path
from . import views

app_name = "core"

urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
    path("contact/", views.contact, name="contact"),
]
```

4.4 URL Parameters

```
from django.urls import path
from . import views

urlpatterns = [
    path("users/<int:user_id>/", views.user_detail, name="user_detail"),
    path("articles/<slug:slug>/", views.article_detail,
name="article_detail"),
]
```

View:

```
from django.http import HttpResponse

def user_detail(request, user_id):
    return HttpResponse(f"User ID: {user_id}")

def article_detail(request, slug):
    return HttpResponse(f"Article slug: {slug}")
```

4.5 Built-in Path Converters

Converter	Meaning
`str`	Any non-empty string excluding `/`
`int`	Positive integer
`slug`	Letters, numbers, hyphens, and underscores
`uuid`	UUID string
`path`	String that may contain `/`

4.6 Named URLs

Use names instead of hard-coding URL paths.

```
path("about/", views.about, name="about")
```

In a template:

```
<a href="{% url 'core:about' %}">About</a>
```

In Python:

```
from django.urls import reverse

url = reverse("core:about")
```

4.7 URL Namespaces

Namespaces prevent URL name collisions between apps.

```
app_name = "core"
```

Then use:

```
{% url 'core:home' %}
```

Common URL Errors

`NoReverseMatch`: Check the URL name, namespace, and required parameters.

404 Not Found: Check the requested path and whether it matches `urlpatterns`.

Chapter 4 Interview Questions

Q1. What is URL routing?

It maps incoming URL paths to views.

Q2. Why use named URLs?

They reduce hard-coded paths and make URL changes easier to manage.

Q3. What does `include()` do?

It includes another URL configuration, commonly from a Django app.

CHAPTER 5: FUNCTION-BASED VIEWS (FBV)

5.1 What Is an FBV?

A Function-Based View is a Python function that accepts an HTTP request and returns an HTTP response.

5.2 Basic View

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Django")
```

5.3 Render a Template

```
from django.shortcuts import render

def home(request):
    context = {
        "title": "Home Page",
        "message": "Welcome to Django",
    }

    return render(request, "core/home.html", context)
```

Template:

```
<h1>{{ title }}</h1>
<p>{{ message }}</p>
```

5.4 Return JSON

```
from django.http import JsonResponse

def api_status(request):
    return JsonResponse({
        "status": "success",
        "message": "API is working",
    })
```

5.5 Get an Object

```
from django.shortcuts import get_object_or_404, render
from .models import Post

def post_detail(request, post_id):
    post = get_object_or_404(Post, pk=post_id)

    return render(
        request,
        "core/post_detail.html",
```

```
        {"post": post},
    )
```

`get_object_or_404()` raises an HTTP 404 response when the object does not exist.

5.6 Handle GET and POST

```
from django.shortcuts import redirect, render

def contact(request):
    if request.method == "POST":
        # Validate submitted data here.
        return redirect("core:home")

    return render(request, "core/contact.html")
```

For real forms, use Django Forms or ModelForms for validation and include CSRF protection.

5.7 Decorators

Decorators can add reusable behavior to views.

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def dashboard(request):
    return render(request, "core/dashboard.html")
```

Advantages of FBVs

- Easy to read
- Explicit control flow
- Good for straightforward views
- Simple to debug

Disadvantages of FBVs

- Shared behavior can be duplicated
- Large views can become difficult to maintain
- Reusable patterns may require extra helper functions

Chapter 5 Interview Questions

Q1. What is a Function-Based View?

A Python function that processes a request and returns a response.

Q2. What is `render()`?

It renders a template with context data and returns an HTTP response.

Q3. What is `redirect()`?

It returns a redirect response to another URL or named view.

CHAPTER 6: CLASS-BASED VIEWS (CBV)

6.1 What Is a CBV?

A Class-Based View defines request-handling behavior in a Python class.

6.2 Basic CBV

```
from django.http import HttpResponse
from django.views import View

class HomeView(View):
    def get(self, request):
        return HttpResponse("Hello from CBV")

    def post(self, request):
        return HttpResponse("POST request received")
```

URL:

```
path("", HomeView.as_view(), name="home")
```

6.3 TemplateView

```
from django.views.generic import TemplateView

class AboutView(TemplateView):
    template_name = "core/about.html"
```

6.4 ListView

```
from django.views.generic import ListView
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = "core/post_list.html"
    context_object_name = "posts"
    paginate_by = 10
```

6.5 DetailView

```
from django.views.generic import DetailView
from .models import Post

class PostDetailView(DetailView):
    model = Post
    template_name = "core/post_detail.html"
```

6.6 CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Post

class PostCreateView(CreateView):
    model = Post
    fields = ["title", "body"]
    template_name = "core/post_form.html"
    success_url = reverse_lazy("core:post_list")
```

6.7 UpdateView

```
from django.urls import reverse_lazy
from django.views.generic import UpdateView
from .models import Post

class PostUpdateView(UpdateView):
    model = Post
    fields = ["title", "body"]
    template_name = "core/post_form.html"
    success_url = reverse_lazy("core:post_list")
```

6.8 DeleteView

```
from django.urls import reverse_lazy
from django.views.generic import DeleteView
from .models import Post

class PostDeleteView(DeleteView):
    model = Post
    template_name = "core/post_confirm_delete.html"
    success_url = reverse_lazy("core:post_list")
```

Security note: For create, update, and delete views, enforce authentication and object-level authorization when required. Never expose unrestricted modification of other users' records.

6.9 FBV vs CBV

FBV	CBV
Defined as a function	Defined as a class
Explicit control flow	Supports reusable class behavior
Often simple to understand	Generic views reduce boilerplate
Shared behavior may require helpers	Inheritance and mixins support reuse

Chapter 6 Interview Questions

Q1. What is a CBV?

A class that defines how a request should be handled.

Q2. Why use `.as_view()`?

It converts a class-based view into a callable view function Django can use in URL routing.

Q3. What is `ListView`?

A generic view for displaying a collection of objects.

Q4. What is `DetailView`?

A generic view for displaying one object.

CHAPTER 7: DJANGO TEMPLATES

7.1 What Is a Template?

A Django template combines HTML with dynamic values and template tags.

Templates help separate presentation from application logic.

7.2 Configure Template Directory

A common project-level structure is:

```
project/
├── templates/
│   └── base.html
├── core/
│   └── templates/
│       └── core/
│           └── home.html
└── manage.py
```

Project-level templates can be configured in `settings.py`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

Keep the standard context processors from your generated project configuration when customizing settings.

7.3 Pass Context to a Template

```
from django.shortcuts import render

def profile(request):
    context = {
        "name": "Omar",
        "role": "Backend Developer",
        "skills": ["Python", "Django", "SQL"],
    }

    return render(request, "core/profile.html", context)
```

7.4 Display Variables

```
<h1>{{ name }}</h1>
<p>{{ role }}</p>
```

7.5 Conditions

```
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}</p>
{% else %}
    <p>Please log in.</p>
{% endif %}
```

7.6 Loops

```
<ul>
    {% for skill in skills %}
        <li>{{ skill }}</li>
    {% empty %}
        <li>No skills found.</li>
    {% endfor %}
</ul>
```

7.7 Template Filters

```
{{ name|lower }}
{{ name|upper }}
{{ description|truncatewords:20 }}
{{ value|default:"Not available" }}
```

7.8 Template Comments

```
{# This comment is not displayed in the rendered page. #}
```

7.9 Security

Django templates automatically escape most variable output.

Avoid using `safe` with untrusted input, because it can allow HTML or script injection.

Chapter 7 Interview Questions

Q1. What is a template?

A file that defines the presentation of a response, often using HTML and Django template syntax.

Q2. What is template context?

A dictionary-like collection of values passed to a template.

Q3. What does `{% for %}` do?

It iterates over a collection.

Q4. What is a template filter?

A filter transforms or formats a value before display.

CHAPTER 8: TEMPLATE INHERITANCE

8.1 Why Use Template Inheritance?

Template inheritance allows multiple pages to reuse a shared layout.

Common elements include:

- Navbar
- Footer
- Sidebar
- CSS links
- JavaScript links
- Main layout

8.2 Create a Base Template

`templates/base.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>

    <header>
        <nav>
            <a href="{% url 'core:home' %}">Home</a>
            <a href="{% url 'core:about' %}">About</a>
        </nav>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>&copy; My Website</p>
    </footer>

</body>
</html>
```

8.3 Extend the Base Template

`templates/core/home.html`

```
{% extends "base.html" %}

{% block title %}
Home Page
{% endblock %}

{% block content %}
<h1>Welcome to Django</h1>
<p>This page extends the base template.</p>
{% endblock %}
```

8.4 Common Template Tags

Tag	Purpose

`{% extends %}`	Inherits another template
`{% block %}`	Defines an overridable section
`{% include %}`	Includes another template
`{% url %}`	Resolves a named URL
`{% load static %}`	Loads static-file template tags
`{% csrf_token %}`	Adds a CSRF token to a form

Chapter 8 Interview Questions

Q1. What is template inheritance?

It allows child templates to reuse and override sections of a base template.

Q2. What does `{% include %}` do?

It includes another template, often for a reusable component.

Q3. What is the benefit of a base template?

It reduces duplication and keeps page layouts consistent.

CHAPTER 9: STATIC FILES AND MEDIA FILES

9.1 Static Files

Static files are application assets such as:

- CSS
- JavaScript
- Logos
- Icons
- Frontend images

9.2 Configure Static Files

In `settings.py`:

```
STATIC_URL = "static/"
```

Example directory:

```
core/
├── static/
│   └── core/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   └── app.js
│       └── images/
│           └── logo.png
```

9.3 Load Static Files in a Template

```
{% load static %}

<link rel="stylesheet" href="{% static 'core/css/style.css' %}">
<script src="{% static 'core/js/app.js' %}"></script>

<img src="{% static 'core/images/logo.png' %}" alt="Logo">
```

9.4 Example CSS

`style.css`

```
body {
    font-family: Arial, sans-serif;
    margin: 20px;
```

```
}

h1 {
    color: navy;
}
```

9.5 Media Files

Media files are user-uploaded files, such as profile pictures and documents.

Install Pillow when using Django's `ImageField`:

```
python -m pip install Pillow
```

Configure development media settings:

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Add development media routing in the project's `urls.py`:

```
from django.conf import settings
from django.conf.urls.static import static
from django.urls import path

urlpatterns = [
    # Your URL patterns
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT,
    )
```

Important: This media-serving pattern is for development. Production uploads should be served through an appropriate storage service or web server.

9.6 ImageField Example

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    image = models.ImageField(upload_to="products/")
```

9.7 Upload Security

- Validate file types and sizes.
- Do not trust filenames or browser-supplied MIME types.
- Avoid executing uploaded files.
- Restrict access to private uploads.
- Consider malware scanning when appropriate.
- Use suitable production storage.

Chapter 9 Interview Questions

Q1. What is the difference between static and media files?

Static files are application assets. Media files are generally uploaded by users.

Q2. What is `STATIC_URL`?

It defines the URL prefix used to reference static assets.

Q3. What is `MEDIA_ROOT`?

It defines the filesystem directory where uploaded media is stored by default.

Q4. Why is `collectstatic` used?

It collects static assets into the configured `STATIC_ROOT` for deployment.

CHAPTER 10: MODELS AND DATABASE

10.1 What Is a Model?

A Django Model is a Python class that defines data structure and database behavior.

Django's ORM uses models to interact with relational databases.

10.2 Create a Model

`core/models.py`

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()
    is_published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
```

```
    def __str__(self):
        return self.title
```

10.3 Common Field Types

Field	Purpose
`CharField`	Short text
`TextField`	Long text
`IntegerField`	Integer values
`PositiveIntegerField`	Non-negative integer values
`DecimalField`	Fixed-precision decimal values
`FloatField`	Floating-point values
`BooleanField`	True/False
`DateField`	Date
`DateTimeField`	Date and time
`EmailField`	Email-format

	validation
`URLField`	URL-format validation
`SlugField`	Slug values
`FileField`	File uploads
`ImageField`	Image uploads
`ForeignKey`	Many-to-one relationship
`OneToOneField`	One-to-one relationship
`ManyToManyField`	Many-to-many relationship

10.4 Important Field Options

```
name = models.CharField(max_length=100)
email = models.EmailField(unique=True)
bio = models.TextField(blank=True)
age = models.PositiveIntegerField(default=18)
```

- `max_length`: Maximum length for fields such as `CharField`.
- `default`: Default value.

- `null=True` : Allows database NULL.
- `blank=True` : Allows an empty value during validation.
- `unique=True` : Enforces uniqueness.
- `choices` : Restricts the permitted choices.
- `db_index=True` : Requests a database index.

Important: `null` controls database nullability; `blank` controls validation.

10.5 Model Metadata

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        verbose_name = "Post"
        verbose_name_plural = "Posts"
```

`Meta` configures model behavior and metadata.

10.6 The `__str__()` Method

```
def __str__(self):
    return self.title
```

This gives model instances a readable string representation, especially in Django admin and the shell.

10.7 Migrations

After creating or modifying models, run:

```
python manage.py makemigrations
python manage.py migrate
```

Review generated migrations when making important schema changes.

10.8 Register the Model in Admin

`core/admin.py`

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
```

```
    list_display = (
        "title",
        "is_published",
        "created_at",
    )
    list_filter = ("is_published",)
    search_fields = ("title", "body")
    ordering = ("-created_at",)
```

10.9 Inspect Models in the Django Shell

```
python manage.py shell
```

Then:

```
from core.models import Post

Post.objects.all()
Post.objects.count()
```

10.10 Model Design Best Practices

1. Give fields meaningful names.
2. Use appropriate field types.
3. Add database constraints when appropriate.
4. Define relationships carefully.
5. Avoid storing duplicate data without a reason.
6. Add indexes based on actual query patterns.
7. Keep models focused.
8. Write tests for important model behavior.
9. Review migrations before deploying.
10. Never assume that frontend validation replaces database constraints.

Chapter 10 Interview Questions

Q1. What is a Django model?

A Python class that defines the structure and behavior of database records.

Q2. What does `__str__()` do?

It returns a readable string representation of an object.

Q3. What is a migration?

A versioned operation that describes and applies database schema changes.

Q4. What is the difference between `blank=True` and `null=True`?

`blank=True` affects validation; `null=True` affects whether the database column may store NULL.

Q5. What does `auto_now_add=True` do?

It sets a date/time field when the object is first created.

Q6. What does `unique=True` do?

It adds a uniqueness constraint so duplicate values are rejected by the database.

END OF PART 1

Completed chapters: 1–10

Next part: Chapters 11–20 — Django ORM, CRUD, Forms, Admin, Authentication, Sessions, Middleware, DRF, Serializers, APIView, Generic Views, and ViewSets.